CSE370: Database Systems
Project Report
Project Title: Disaster Relief Management System

Group No: 8, CSE370 Lab Section: 11, Summer 2026		
ID	**Name**	**Contribution**
24301374	Shadeed Aarshan	
24301362	Nuzhat Tabassum Oishee	
24301348	Kazi Aryan Rahman	

Table of Contents

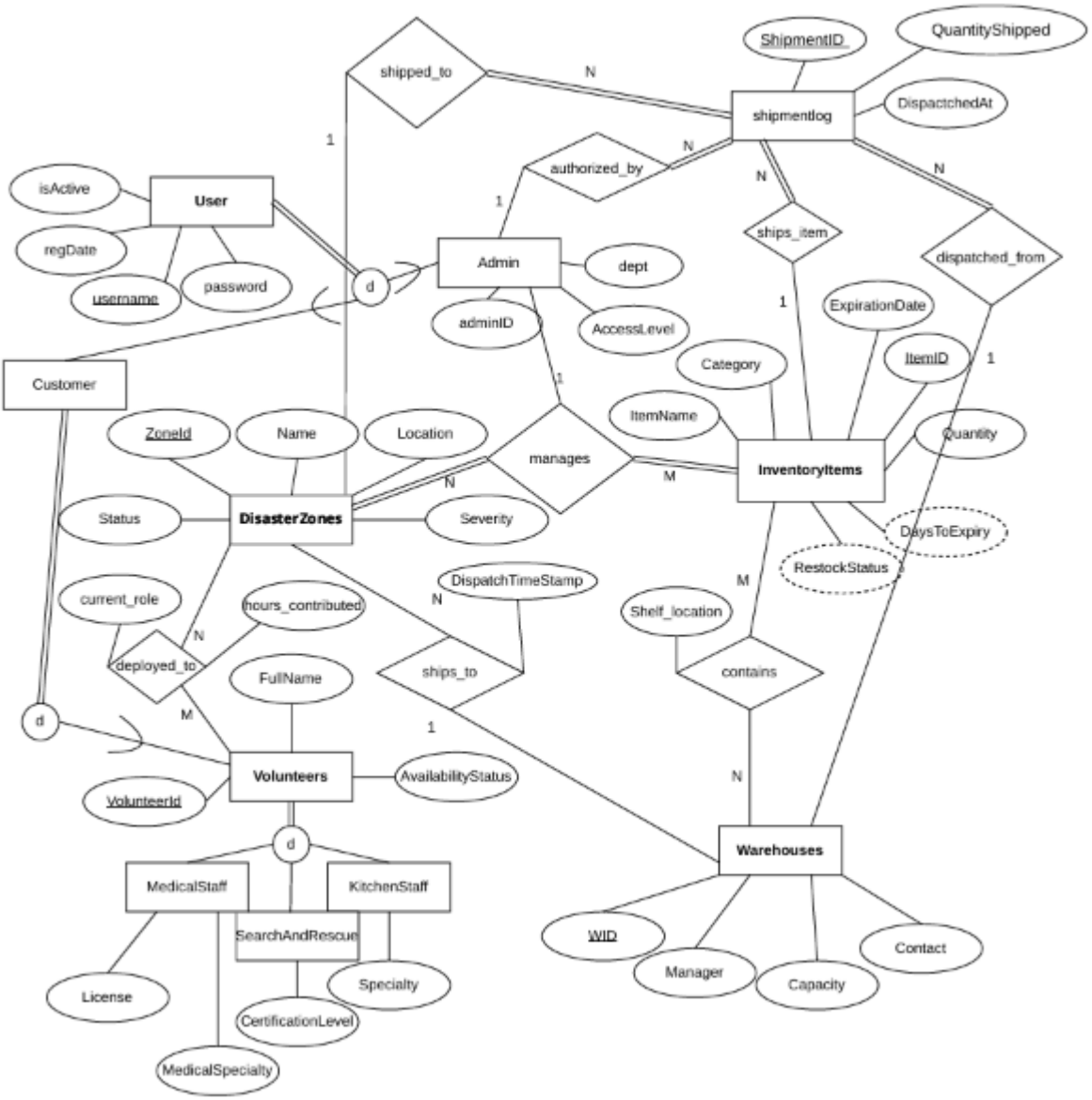
Introduction

The idea behind our DBMS project is “Disaster Relief System” which takes a real life problem of civilians suffering from natural disasters and creates a prototype of a solution that can manage the relief system during these times. Our system will allow users to view disaster zones and sign up as volunteers to help victims.

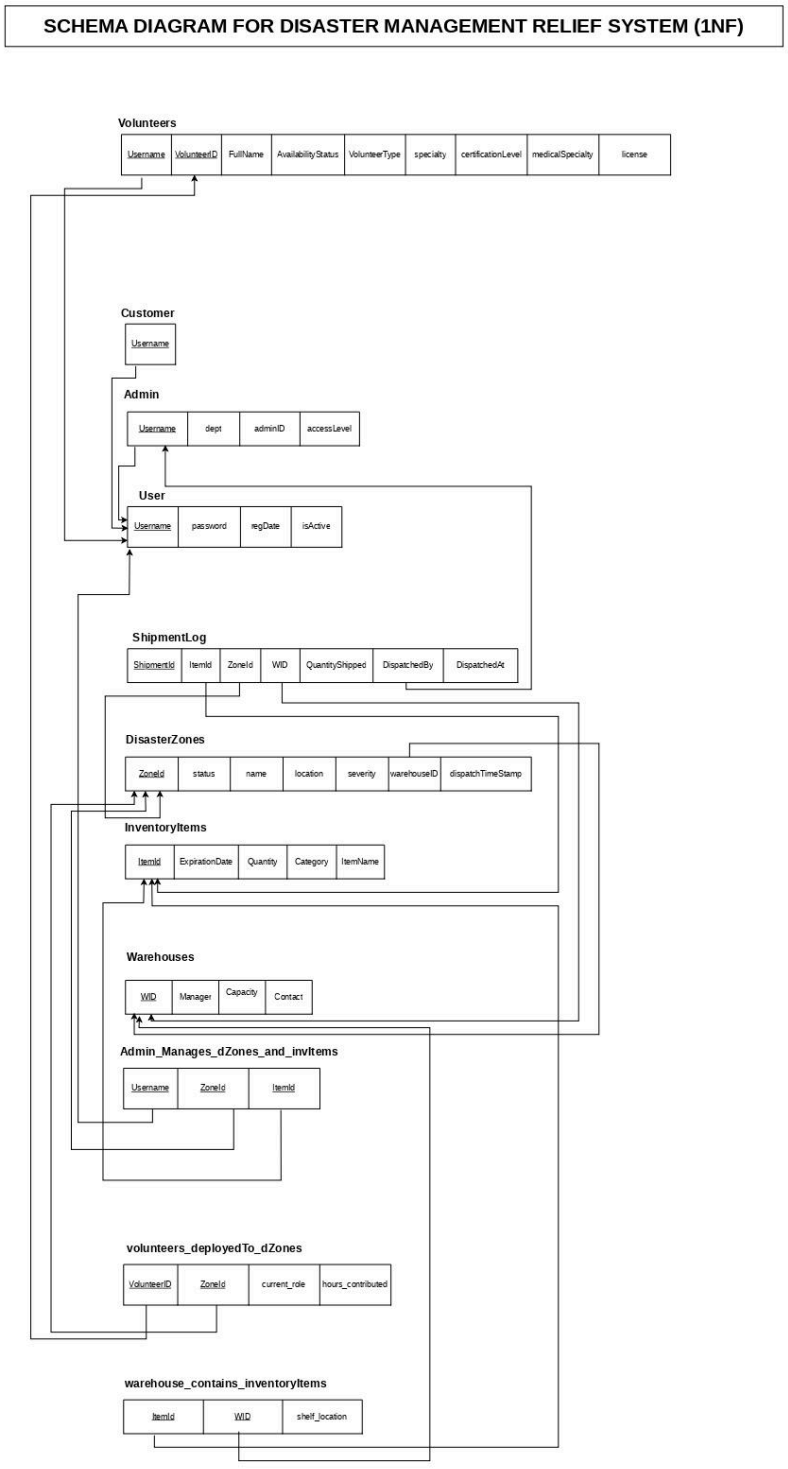
Project Features

ID, Name	Features	
	Ft 1	Login
24301374, Shadeed Aarshan	Ft 2	Customers can view disaster zones and apply to be volunteers
	Ft 3	Admin can approve/delete volunteers
	Ft 4	Admin can deploy/release volunteers to and from disaster zones
	Ft 5	Volunteers can toggle availability, controlling whether they can be deployed or not
24301348, Kazi Aryan Rahman	Ft 1	Admin can declare, view, and update disaster zones, assigning a severity level and source warehouse
	Ft 2	Admin can dispatch relief supplies from a warehouse to a disaster zone, with automatic stock deduction and shipment logging
	Ft 3	Admin can view live per-zone analytics — active personnel, accumulated hours, and supplies dispatched
23101997, DEF	Ft 1	Warehouse Management and Inventory Assignment: Register warehouses, view/remove/edit warehouses and shelf locations
	Ft 2	Inventory Stock Monitoring and Low Stock Management: view inventory items, add/remove/update inventory
	Ft 3	Expiry Date Auditor: view remaining days until expiry, flag expired, and soon-to-be expired items

ER/EER Diagram



Schema Diagram



Normalization

a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

Answer:

a. During schema mapping, it is automatically converted to 1NF as there are no multivalued or composite attributes
b. Yes, it is in 2NF because there are no functional dependencies
c. Yes, it is in 3NF because there are no transitive dependencies

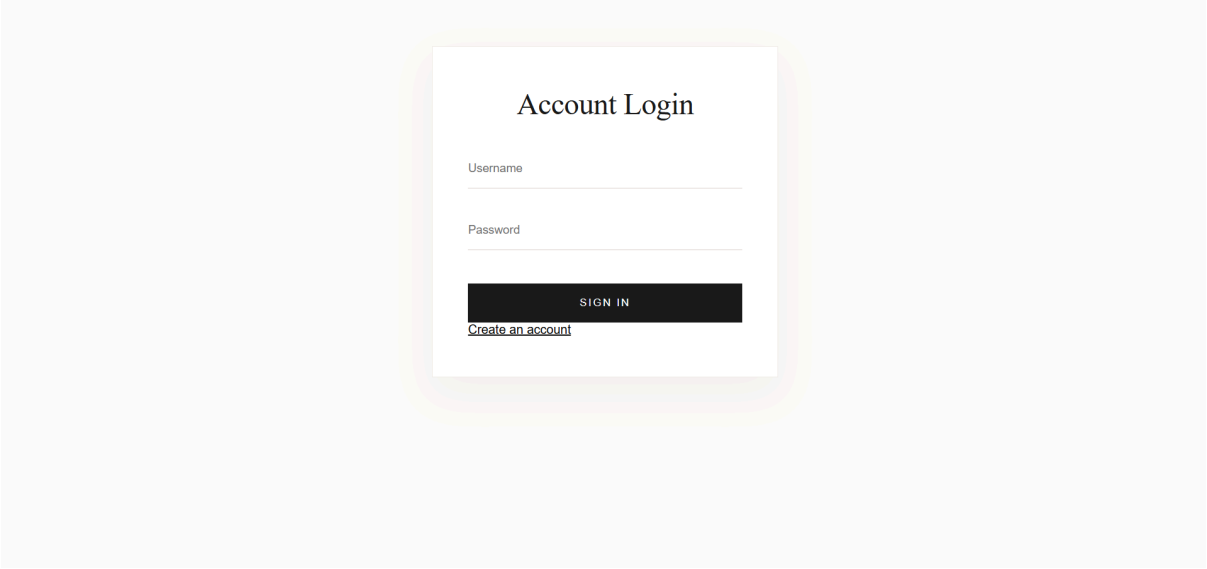
Frontend Development

Briefly discuss about Backend Development and add relevant Screenshots (if required) by mentioning Individual Contributions

Contribution of ID: 24301374, Name: Shadeed Aarshan

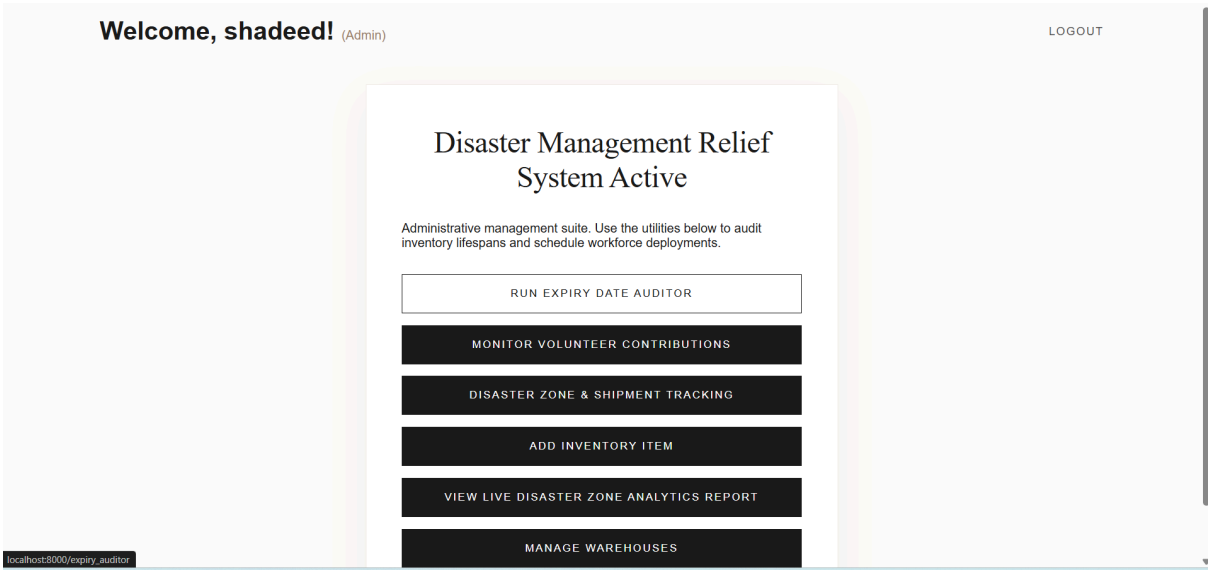
In the backend section, I designed the following pages:

Login page:

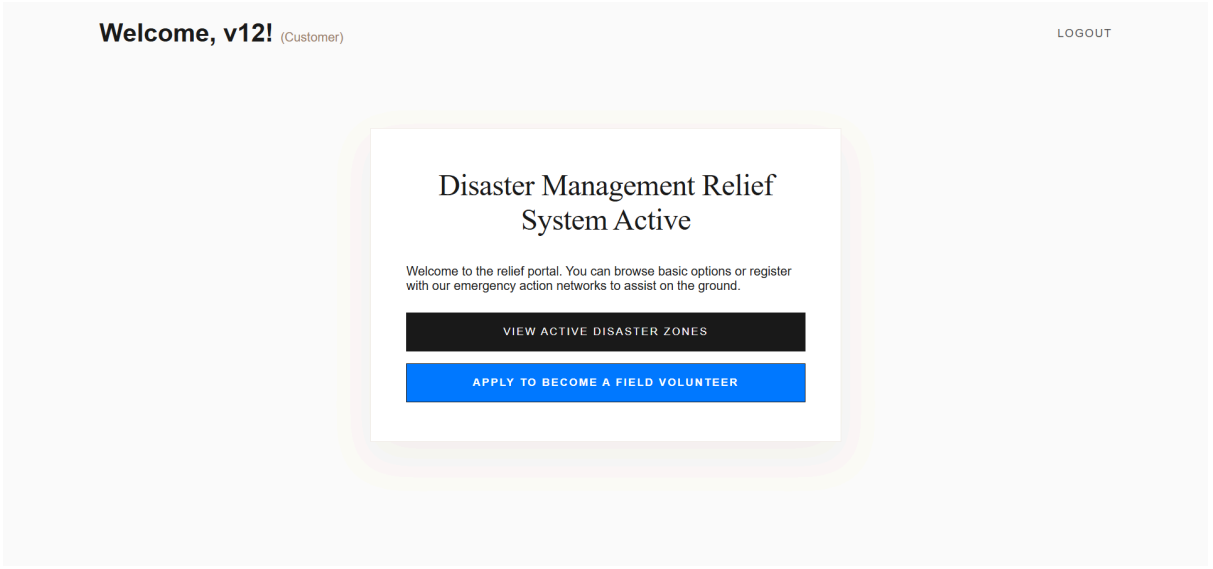


I contributed in designing the admin home page alongside my groupmates

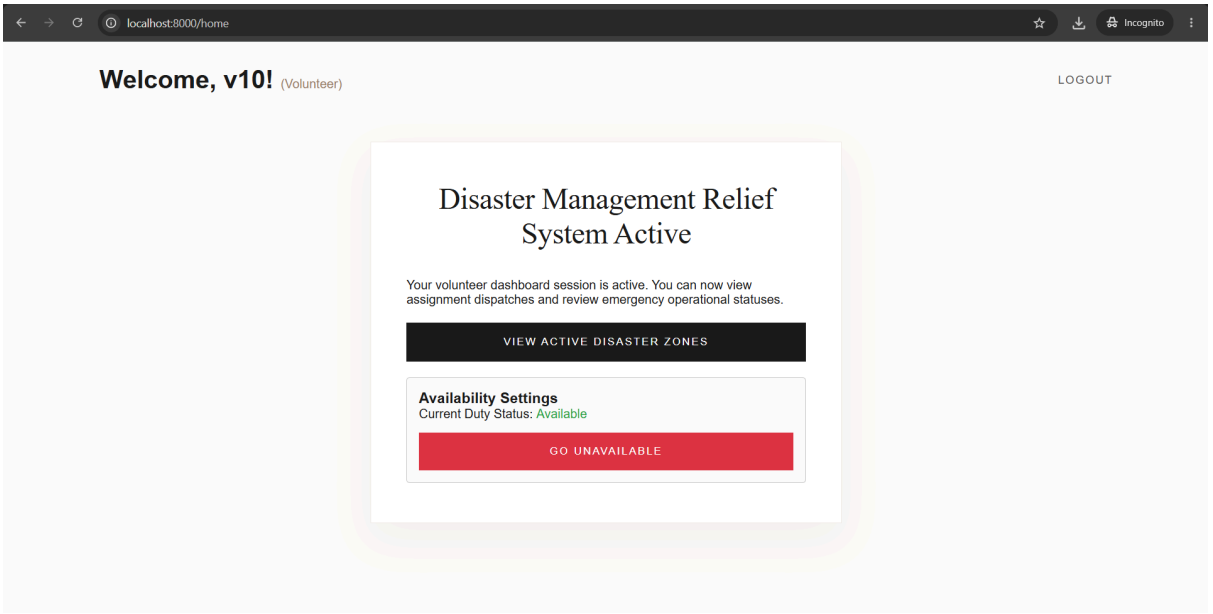
Home page(if admin): admin personnel can navigate their various features



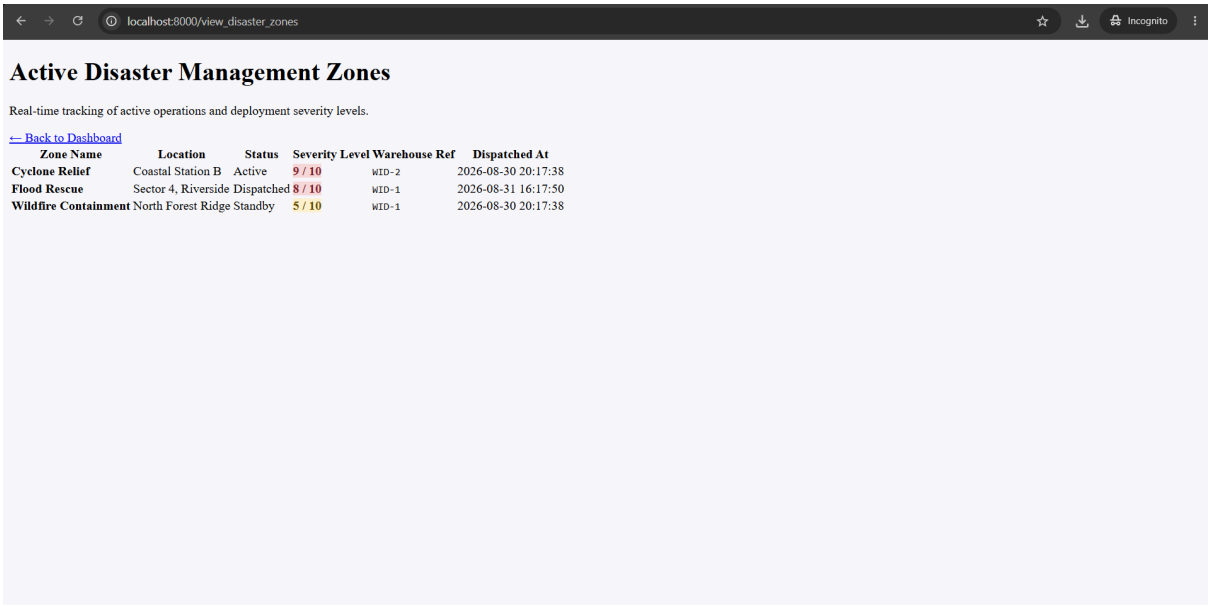
If customer but not volunteer: customers can choose to view active disaster zones or apply to become a volunteer.



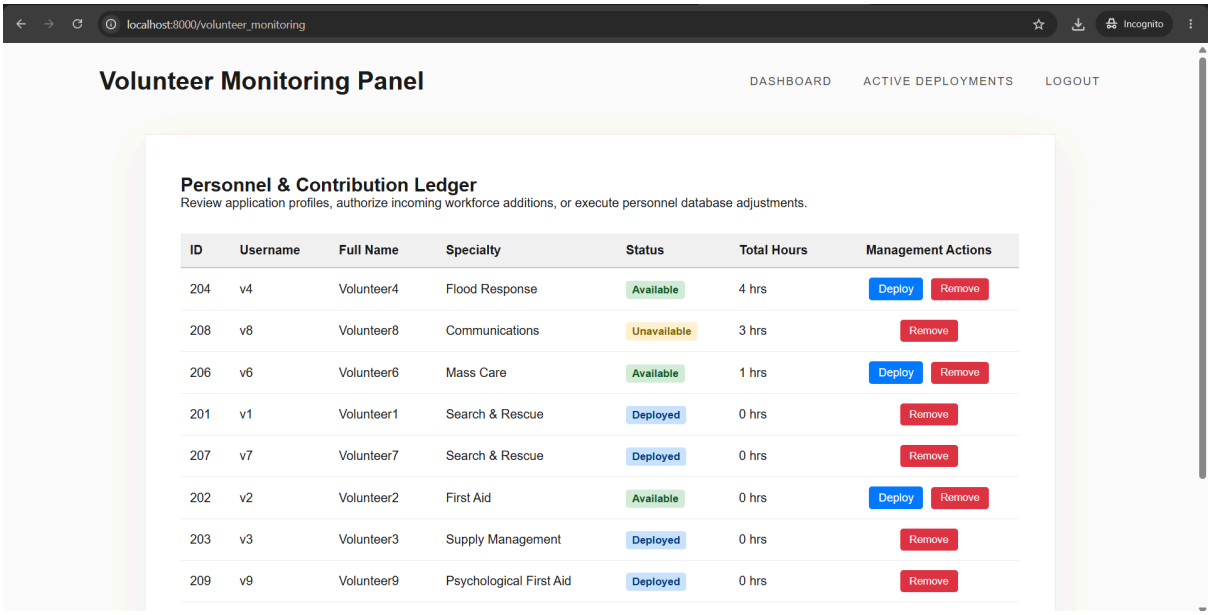
If volunteer: Volunteers can toggle their availability status.



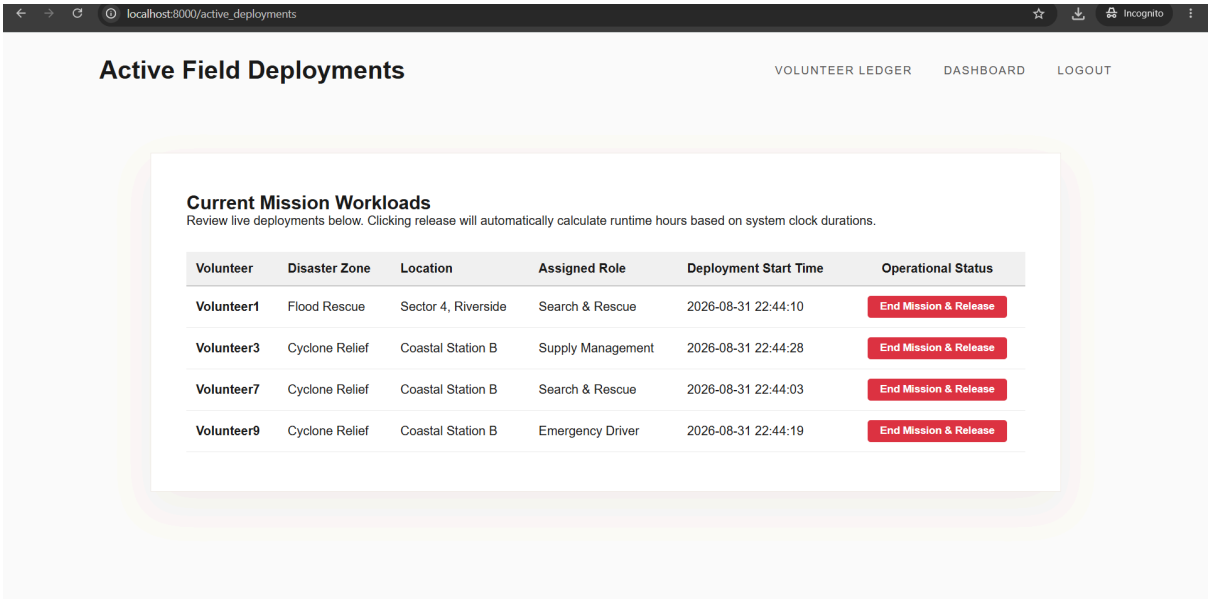
View_disaster_zones page:



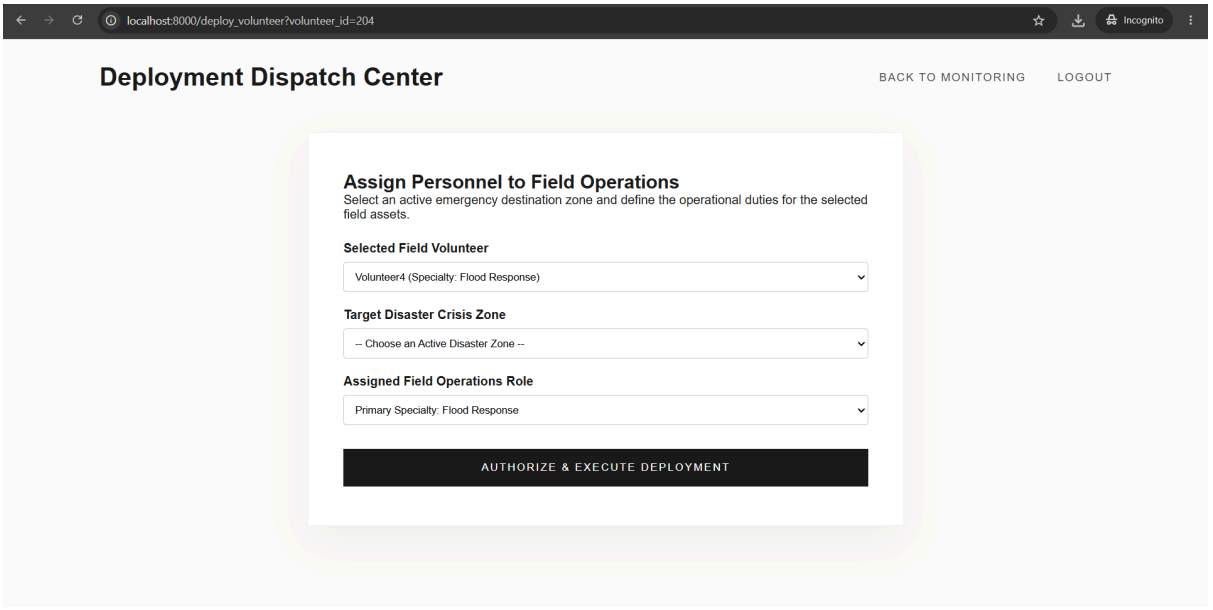
Volunteer_monitoring: Admin personnel can monitor volunteers, approve/remove them, see who is available and deploy them. They can also view active deployments.



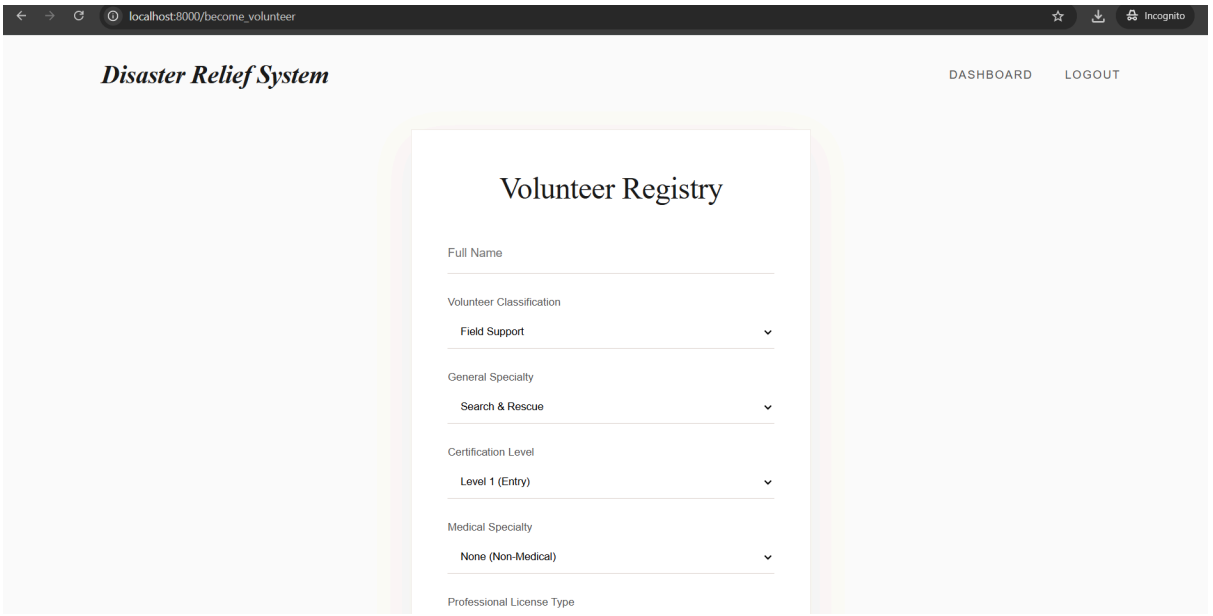
Active_deployments: Admin personnel can view current deployments. They can also release the volunteers from deployment.



Deploy_volunteer: Here, admin can choose the options and details regarding deploying a volunteer.



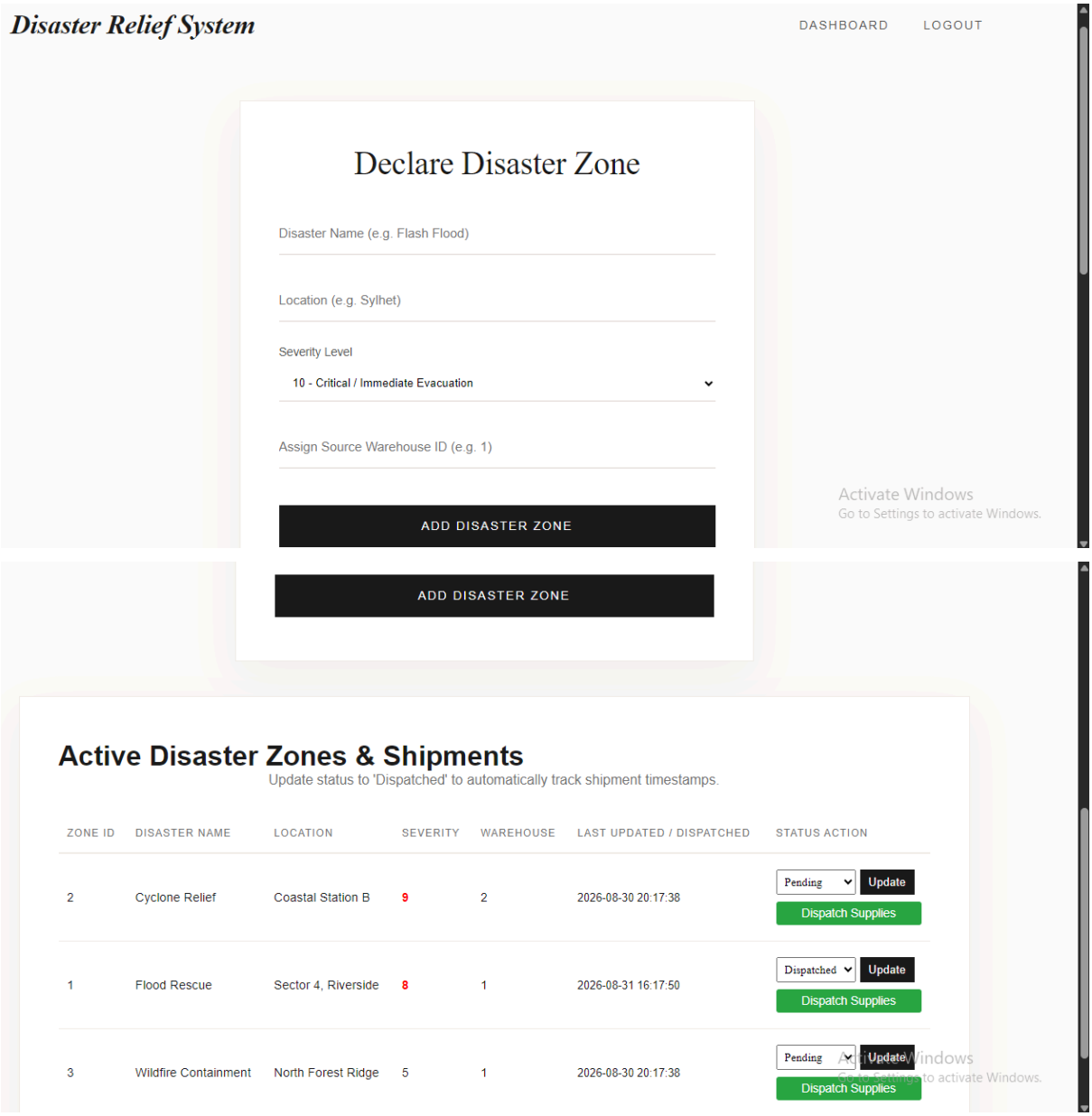
Become_volunteer: Menu for customers to become a volunteer by providing relevant information.



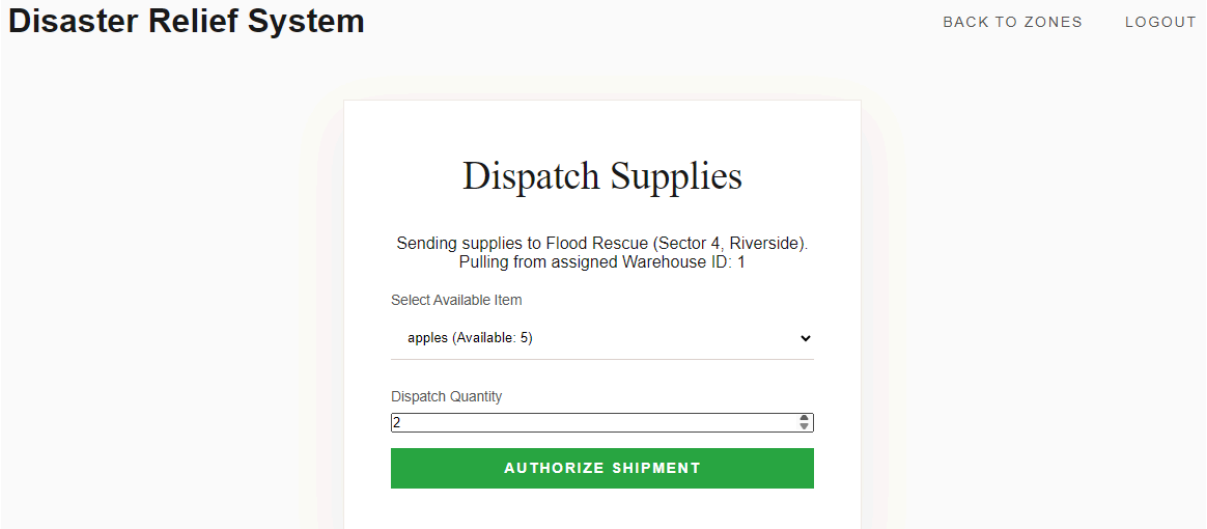
Contribution of ID: 24301348, Name: Kazi Aryan Rahman

In the frontend section, I designed the following administrative interfaces:

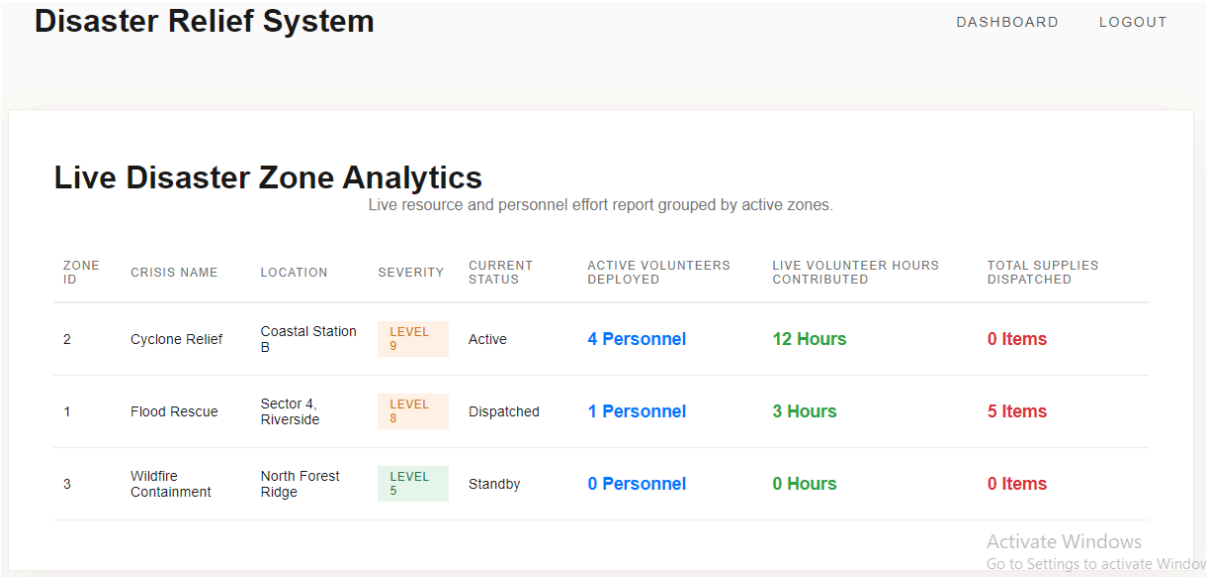
1. manage_zones page (Disaster Zone & Shipment Tracking): Interface for admins to declare new disaster zones and update the operational statuses of active ones.



2. dispatch_shipment Page: Interface to validate and select available inventory from an assigned warehouse to dispatch to a specific crisis zone.



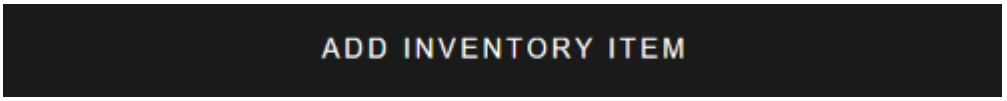
3. zone_analytics Page: Dashboard displaying a real-time, aggregated report on active volunteer deployments and total dispatched supplies per zone.

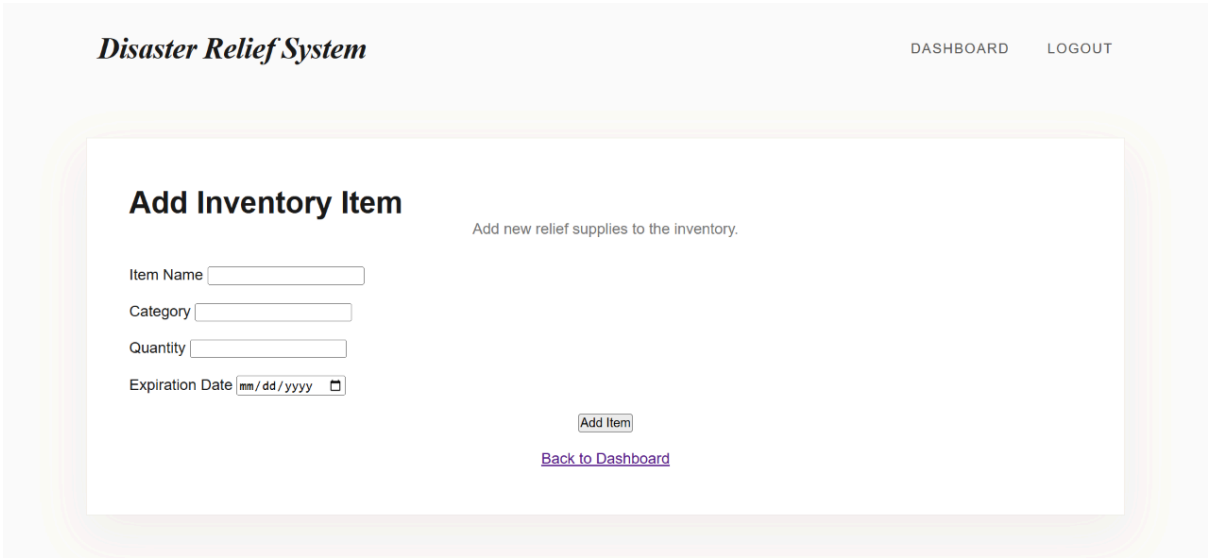


Contribution of ID: 24301362, Name: Nuzhat Tabassum Oishee

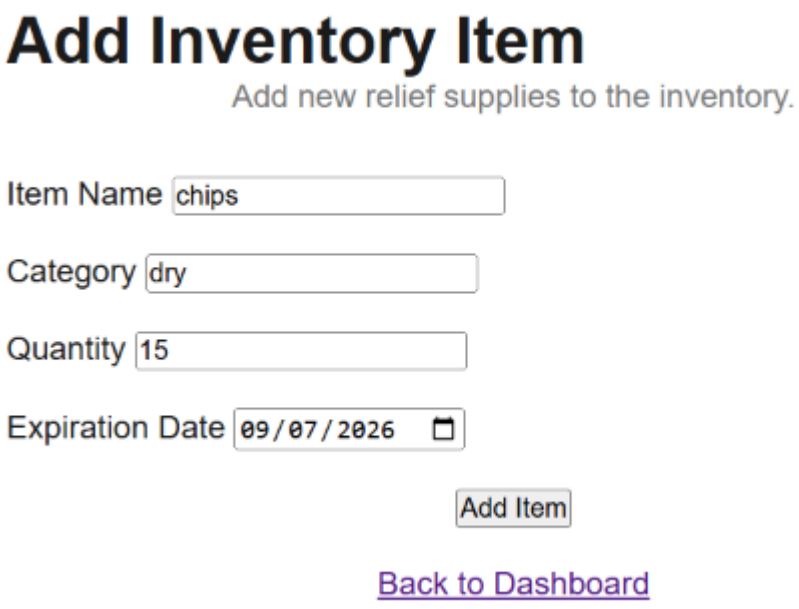
In the frontend section, I designed the following pages:

1. **Add_inventory page**
   This page adds an inventory item (which is seen in the expiry date auditor after being added) based on item name, category, quantity, and expiration date





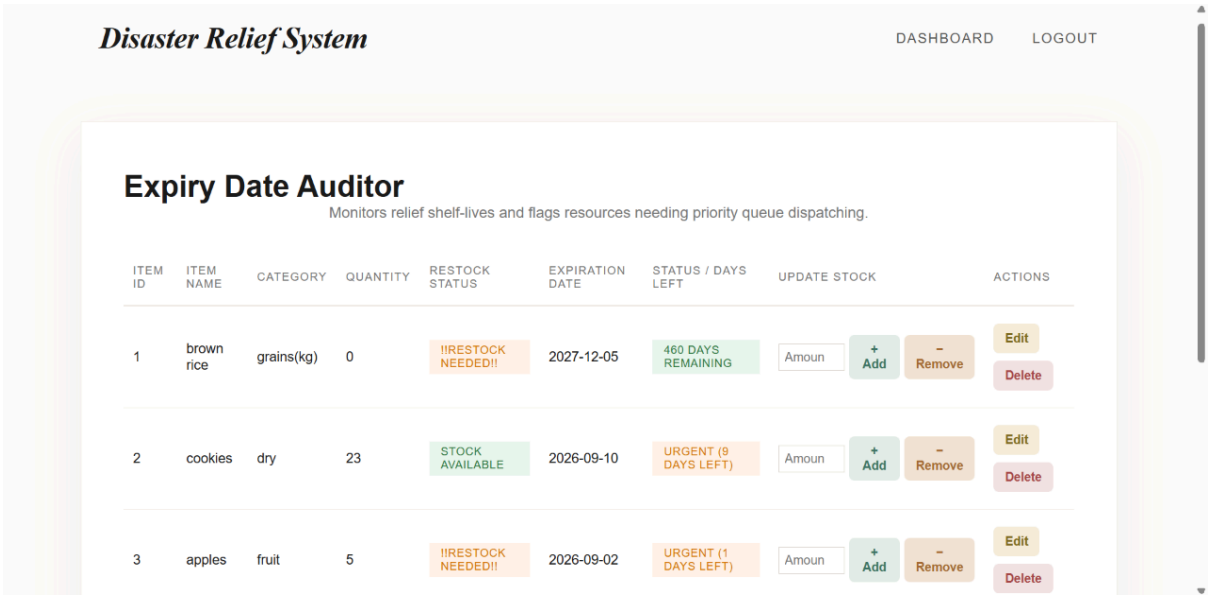
Adding an item, eg., chips here, will cause it to show up in the expiry date auditor



What we see in the expiry date auditor after adding it:

2. Expiry date auditor page

The goal of this page is to view the added inventory items based on their shelf lives. We can see every inventory item's name, category, quantity, expiration date and days left to expire. Moreover, based on their quantity, it tells us whether a restock is needed or not.



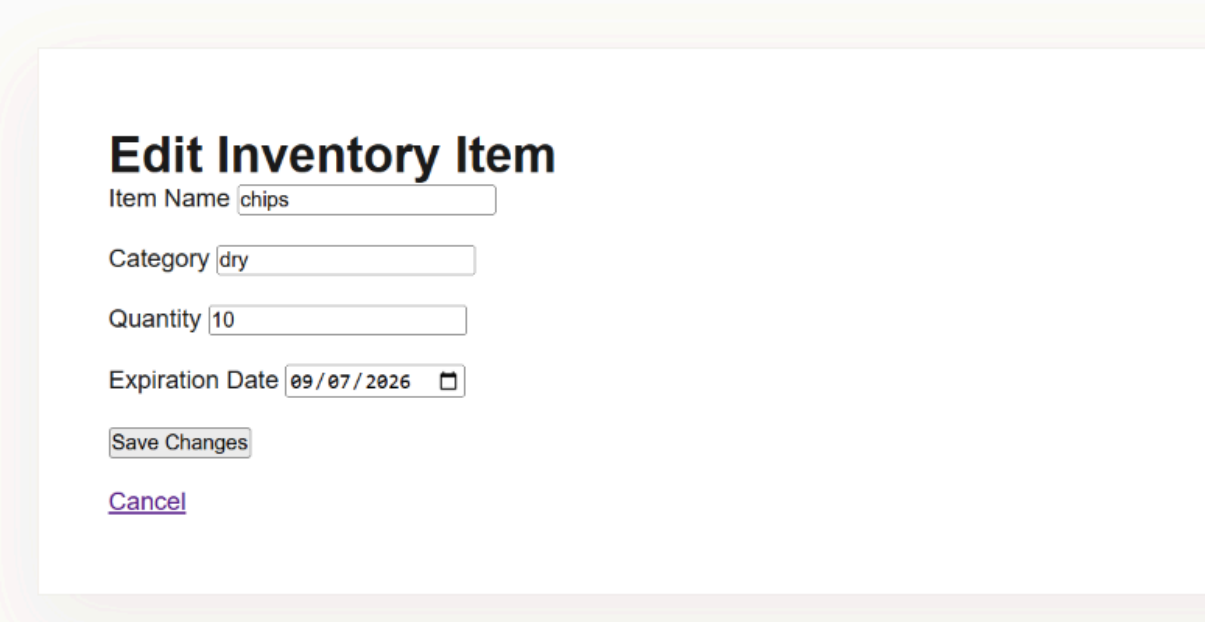
There's also an option to update, add, or remove stock quantities. Eg.
Previously we had this

If we want to remove 5 of this item, then

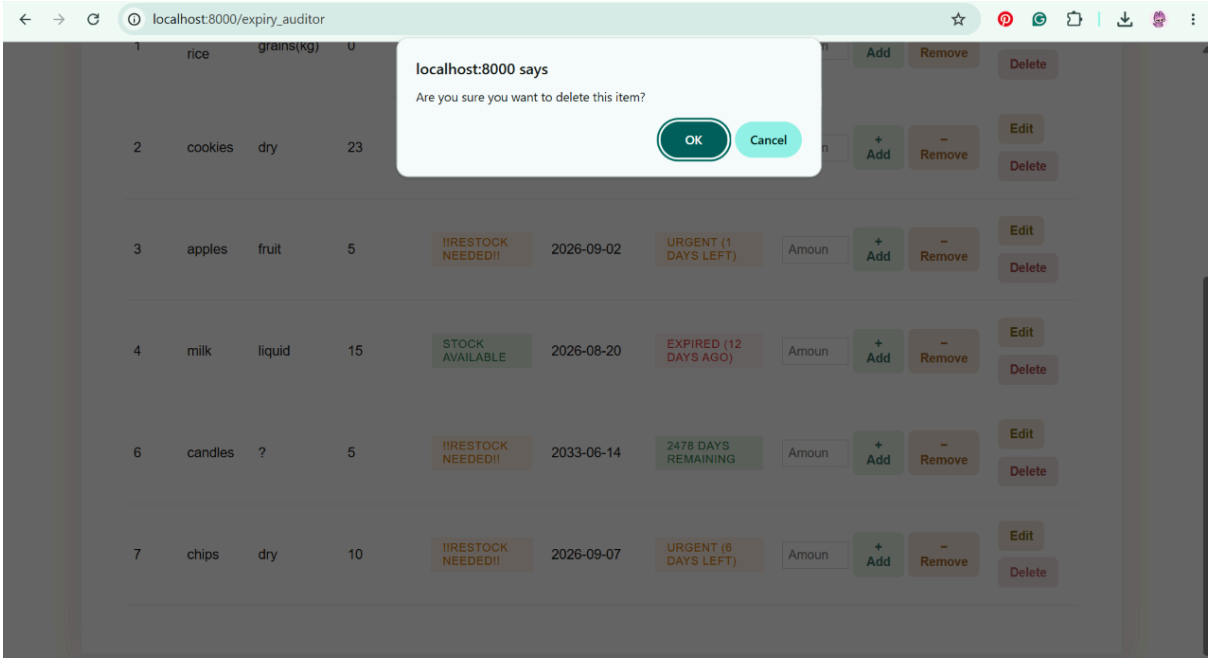
After updating, we can see

Additionally, we can edit or delete inventory items using the edit and delete buttons at the rightmost corner

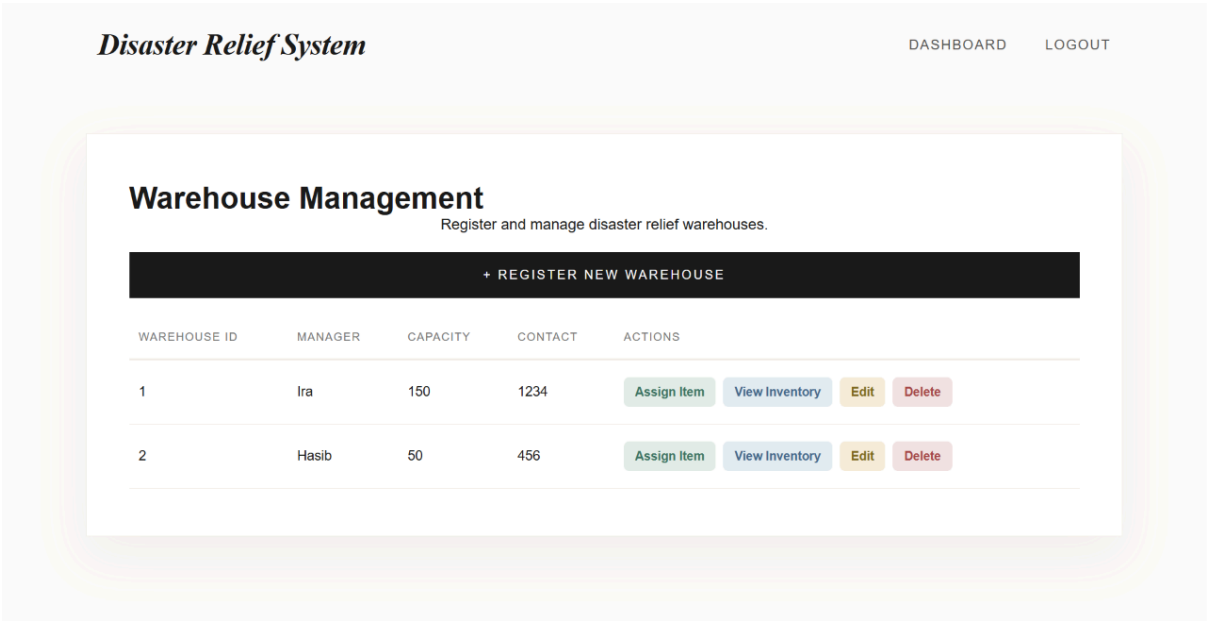
Edit inventory page:



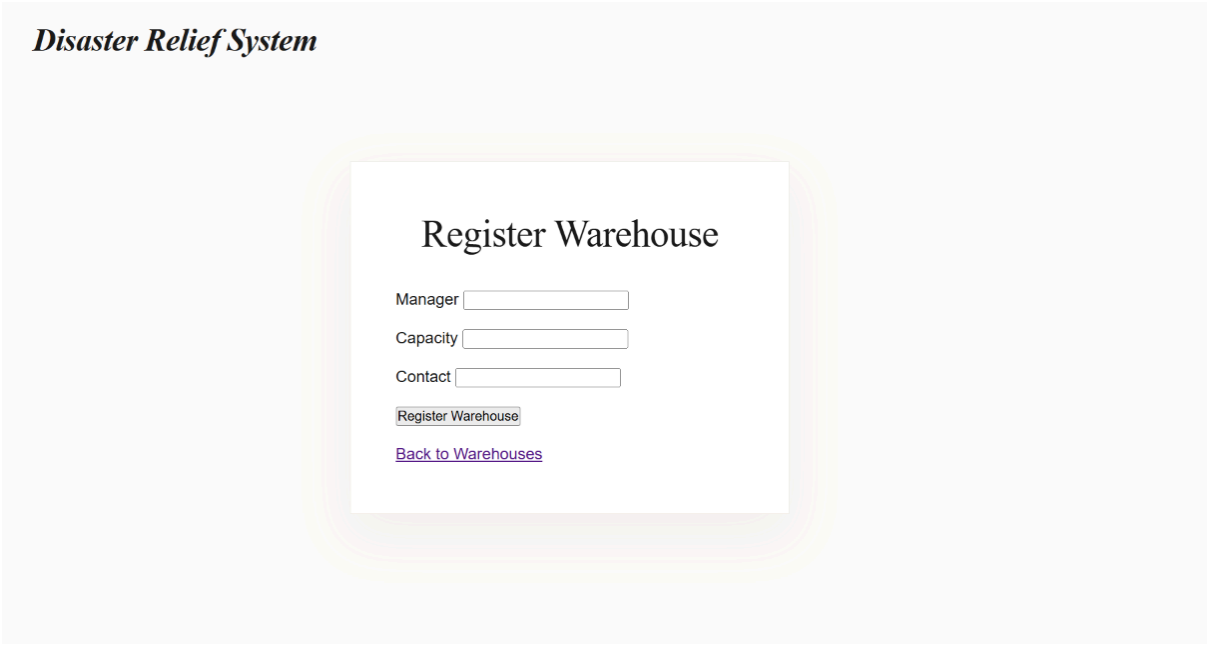
If we want to delete, then this shows up



3. **Manage warehouse page**
   This page allows us to register warehouses, view the registered warehouses, assign inventory items to a warehouse, view a warehouse's inventory stock, edit and delete a warehouse.

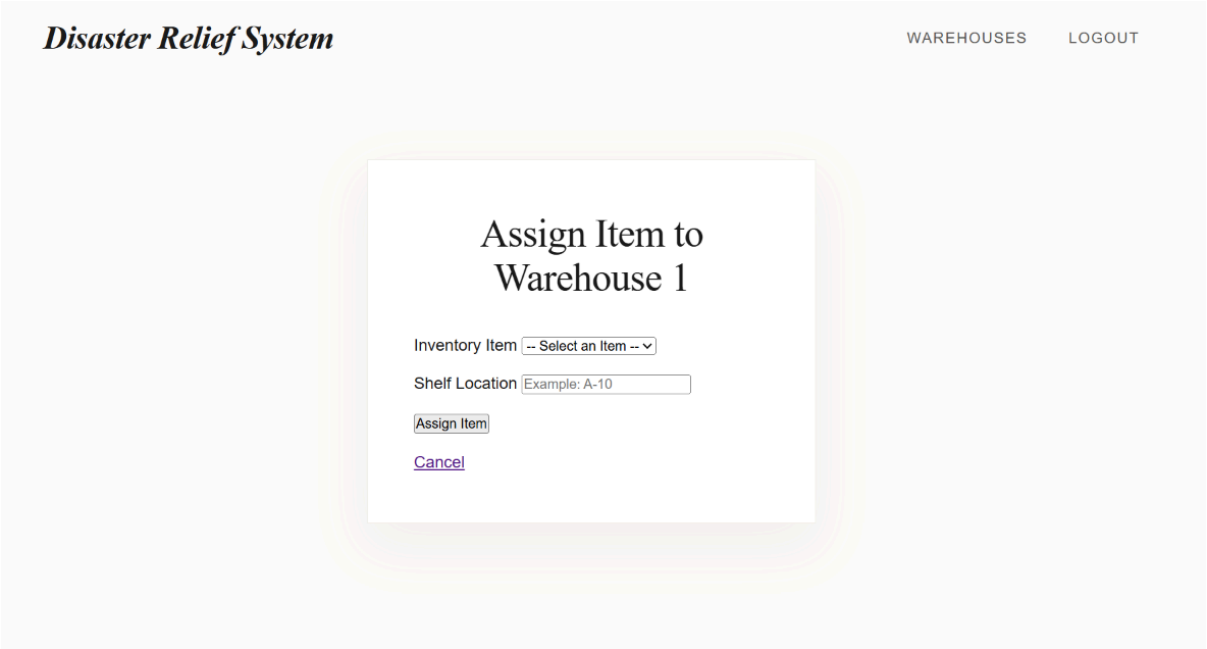


Registering a warehouse:

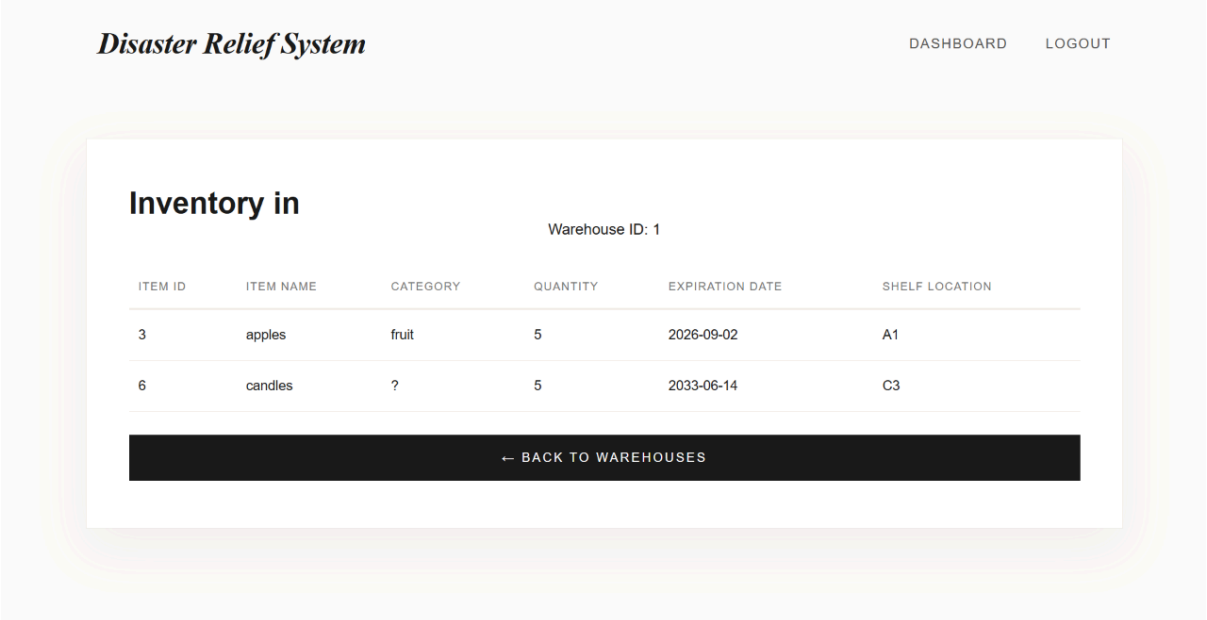


If we want to add a new warehouse, we have to register it first. Then it will show up in the warehouse management panel.

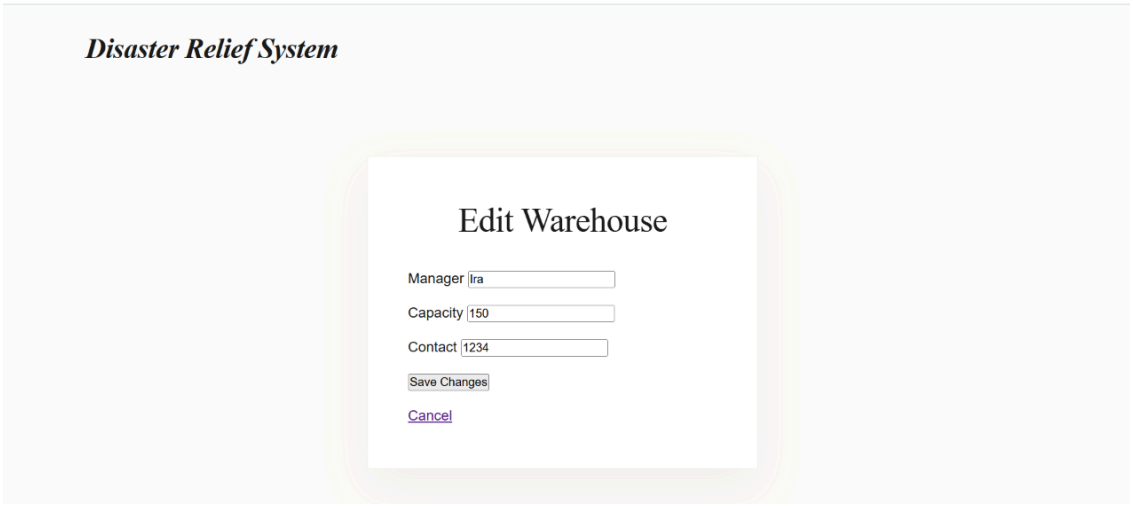
After that, we can assign items to that warehouse by selecting available inventory items and designating them to a shelf location.



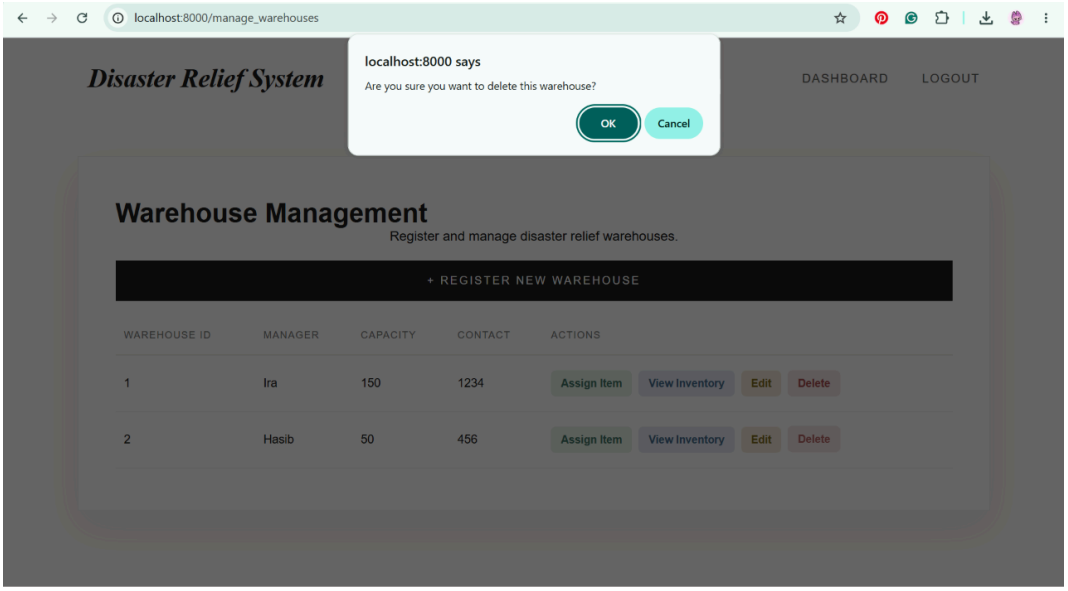
We can also view the inventory of a warehouse:



Editing a warehouse:



Deleting a warehouse:



Backend Development

Briefly discuss about Backend Development and add relevant Screenshots (if required) by mentioning Individual Contributions

Contribution of ID : 24301374, Name : Shadeed Aarshan

We used Python with Flask for our project. I developed the login route for customers to enter their username and password. Following is the query running in the login route

```
sql = "SELECT * FROM User WHERE username = %s AND password = %s AND isActive = 1"
cursor.execute(sql, (username, password))
user = cursor.fetchone()

if user:
    cursor.execute("SELECT * FROM Admin WHERE Username = %s", (username,))
    is_admin = cursor.fetchone()

    cursor.execute("SELECT * FROM Customer WHERE Username = %s", (username,))
    is_customer = cursor.fetchone()

    session['username'] = username
    if is_admin:
        session['role'] = 'Admin'
    elif is_customer:
        cursor.execute("SELECT * FROM Volunteers WHERE Username = %s", (username,))
        if cursor.fetchone():
            session['role'] = 'Volunteer'
        else:
            session['role'] = 'Customer'
    else:
        session['role'] = 'User'
```

I developed the feature for customers to be able to apply as volunteers. This updates the volunteer table. Following is the query in the /become_volunteer route

```
        try:
            sql = """
                INSERT INTO Volunteers
(Username, FullName, AvailabilityStatus,
VolunteerType, specialty, certificationLevel,
medicalSpecialty, license)
                VALUES (%s, %s, 'Pending', %s,
%s, %s, %s, %s)
            """
            cursor.execute(sql,
(session['username'], full_name, vol_type,
specialty, cert_level, med_specialty,
license_num))
            conn.commit()

            return
```

Admin can monitor volunteers through the /monitor_volunteer route

```
    sql = """
        SELECT v.VolunteerID, v.Username,
v.FullName, v.specialty, v.AvailabilityStatus,
               IFNULL(history_counts.hrs, 0) AS
total_hours
        FROM Volunteers v
        LEFT JOIN (
            SELECT VolunteerID,
SUM(hours_earned) as hrs
            FROM deployment_history GROUP BY
VolunteerID
        ) history_counts ON v.VolunteerID =
history_counts.VolunteerID
        ORDER BY field(v.AvailabilityStatus,
'Pending') DESC, total_hours DESC
    """
```

Connecting with the previous feature, I developed the admin feature of approving/removing volunteers. Following pictures are the queries ran by the /approve_volunteer and /remove_volunteer routes respectively.

/approve_volunteer

```
    try:
        cursor.execute("UPDATE Volunteers SET
AvailabilityStatus = 'Available' WHERE
VolunteerID = %s", (volunteer_id,))
        conn.commit()
```

/remove_volunteer

```
    try:
        cursor.execute("DELETE FROM
volunteers_deployedto_dzones WHERE VolunteerID =
%s", (volunteer_id,))
        cursor.execute("DELETE FROM Volunteers
WHERE VolunteerID = %s", (volunteer_id,))
        conn.commit()
```

I developed the admin feature of being able to deploy/release volunteers to and from disaster zones. Admin can also view current deployments.

/deploy_volunteers

```
    cursor.execute("SELECT VolunteerID,
FullName, specialty FROM Volunteers WHERE
AvailabilityStatus = 'Available'")
    available_vols = cursor.fetchall()

    cursor.execute("SELECT ZoneId, name,
location, severity FROM disasterzones")
    zones = cursor.fetchall()

    user_specialty = ""
    if selected_volunteer_id:
        cursor.execute("SELECT specialty FROM
Volunteers WHERE VolunteerID = %s",
(selected_volunteer_id,))
        result = cursor.fetchone()
        if result:
            user_specialty = result['specialty']
```

/process_deployment

```
    try:
        sql_deploy = """
            INSERT INTO
volunteers_deployedto_dzones (`VolunteerID`,
`ZoneId`, `current_role`, `hours_contributed`,
`deployed_at`)
            VALUES (%s, %s, %s, 0, NOW())
        """
        cursor.execute(sql_deploy,
(volunteer_id, zone_id, role))
        cursor.execute("UPDATE Volunteers SET
AvailabilityStatus = 'Deployed' WHERE
VolunteerID = %s", (volunteer_id,))
        conn.commit()
        return
```

/active_deployments

```
    sql = """
        SELECT d.VolunteerID, d.ZoneId,
d.current_role, d.hours_contributed,
d.deployed_at,
               v.FullName, dz.name AS zone_name,
dz.location
        FROM volunteers_deployedto_dzones d
        JOIN Volunteers v ON d.VolunteerID =
v.VolunteerID
        JOIN disasterzones dz ON d.ZoneId =
dz.ZoneId
    """
    cursor.execute(sql)
    deployments = cursor.fetchall()
```

/release_volunteer

```
    try:
        cursor.execute("SELECT deployed_at FROM
volunteers_deployedto_dzones WHERE VolunteerID =
%s AND ZoneId = %s", (volunteer_id, zone_id))
        deployment = cursor.fetchone()

        if deployment:
            cursor.execute("SELECT
TIMESTAMPDIFF(HOUR, %s, NOW()) AS hours_spent",
(deployment['deployed_at'],))
            result = cursor.fetchone()
            calculated_hours =
result['hours_spent'] if result and
result['hours_spent'] is not None else 0
            if calculated_hours < 1:
                calculated_hours = 1

            cursor.execute("""
                INSERT INTO deployment_history
(VolunteerID, hours_earned)
                VALUES (%s, %s)
            """, (volunteer_id,
calculated_hours))

            cursor.execute("DELETE FROM
volunteers_deployedto_dzones WHERE VolunteerID =
%s AND ZoneId = %s", (volunteer_id, zone_id))
            cursor.execute("UPDATE Volunteers
SET AvailabilityStatus = 'Available' WHERE
VolunteerID = %s", (volunteer_id,))

        conn.commit()
        return
```

Finally, I developed the customer feature to be able to toggle their availability. If they are unavailable, they cannot be deployed.

/volunteer/toggle_availability

```
    try:
        cursor.execute("UPDATE Volunteers SET
AvailabilityStatus = %s WHERE Username = %s AND
AvailabilityStatus != 'Deployed'", (new_status,
username))
        conn.commit()
```

Contribution of ID : 24301348, Name : Kazi Aryan Rahman

I developed the Disaster Zone & Shipment Tracking feature, which lets admins declare new disaster zones and assign them a severity level and a source warehouse. Zones are listed ordered by severity so the most critical ones surface first, and admins can update a zone's operational status as relief progresses.

/manage_zones

```
#based on severity highest one is shown
cursor.execute("SELECT * FROM disasterzones ORDER BY severity DESC")
zones = cursor.fetchall()
```

/add_zone

```
cursor.execute("SELECT MAX(ZoneId) FROM disasterzones")
max_id = cursor.fetchone()[0]
new_id = 1 if max_id is None else max_id + 1

sql = """
    INSERT INTO disasterzones (ZoneId, name, location, severity, warehouseID, status)
    VALUES (%s, %s, %s, %s, %s, %s)
"""
cursor.execute(sql, (new_id, name, location, severity, warehouse_id, status))
conn.commit()
```

/update_zone/<zone_id>

```
cursor.execute("UPDATE disasterzones SET status = %s WHERE ZoneId = %s", (new_status, zone_id))
conn.commit()
```

Building on that, I developed the Relief Shipment Dispatcher. It pulls the items currently available in a zone's assigned warehouse, then on submission checks that enough stock exists, deducts the dispatched quantity from inventory, records the dispatch in a permanent shipment log, and automatically updates the zone's status to 'Dispatched' with a timestamp.

/dispatch_shipment/<zone_id> — GET (load items available in the zone's warehouse)

```
fetch_inventory_sql = """
    SELECT i.ItemId, i.ItemName, i.Quantity, i.Category
    FROM inventoryitems i
    JOIN warehouse_contains_inventoryitems wci ON i.ItemId = wci.ItemId
    JOIN warehouses w ON w.WID = wci.WID
    JOIN disasterzones dz ON dz.warehouseID = w.WID
    WHERE dz.ZoneId = %s AND i.Quantity > 0
"""
cursor.execute(fetch_inventory_sql, (zone_id,))
available_items = cursor.fetchall()
```

/dispatch_shipment/<zone_id> — POST (validate stock, deduct inventory, log shipment, update zone status)

```
# check if enough stock exists first
cursor.execute("SELECT Quantity FROM inventoryitems WHERE ItemId = %s", (item_id,))
item_data = cursor.fetchone()

if not item_data or item_data['Quantity'] < dispatch_qty:
    return "Error: Insufficient stock for this dispatch."

# get the assigned warehouse ID for this zone
cursor.execute("SELECT warehouseID FROM disasterzones WHERE ZoneId = %s", (zone_id,))
zone_data = cursor.fetchone()
warehouse_id = zone_data['warehouseID'] if zone_data else 0

# deduct from inventory
update_sql = "UPDATE inventoryitems SET Quantity = Quantity - %s WHERE ItemId = %s"
cursor.execute(update_sql, (dispatch_qty, item_id))

# log the shipment dispatch into our new table
log_sql = """
    INSERT INTO shipmentlog (ItemId, ZoneId, WID, QuantityShipped, DispatchedBy, DispatchedAt)
    VALUES (%s, %s, %s, %s, %s, NOW())
"""
cursor.execute(log_sql, (item_id, zone_id, warehouse_id, dispatch_qty, session['username']))

# automatically update zone status to dispatched
cursor.execute(
    "UPDATE disasterzones SET status = 'Dispatched', dispatchTimeStamp = NOW() WHERE ZoneId = %s",
    (zone_id,)
)

conn.commit()
```

Finally, I developed the Live Disaster Zone Analytics & Resource Report — a single aggregation query joining zones with their deployed volunteers and shipment log to report, per zone, active personnel, live accumulated volunteer hours, and total supplies dispatched, ordered by severity.

/zone_analytics

```
analytics_sql = """
    SELECT
        dz.ZoneId,
        dz.name,
        dz.location,
        dz.severity,
        dz.status,
        COUNT(DISTINCT vd.VolunteerID) AS active_personnel,
        COALESCE(SUM(TIMESTAMPDIFF(HOUR, vd.deployed_at, NOW())), 0) AS live_hours_contributed,
        COALESCE(sl.total_shipped, 0) AS total_items_dispatched
    FROM disasterzones dz
    LEFT JOIN volunteers_deployedto_dzones vd ON dz.ZoneId = vd.ZoneId
    LEFT JOIN (
        SELECT ZoneId, SUM(QuantityShipped) AS total_shipped
        FROM shipmentlog GROUP BY ZoneId
    ) sl ON dz.ZoneId = sl.ZoneId
    GROUP BY dz.ZoneId, dz.name, dz.location, dz.severity, dz.status, sl.total_shipped
    ORDER BY dz.severity DESC, active_personnel DESC
"""
cursor.execute(analytics_sql)
analytics = cursor.fetchall()
```

Contribution of ID: 24301362, Name: Nuzhat Tabassum Oishee

I developed the features that handled the viewing, adding, editing, and deleting of inventory items. To add an item to the inventory:

```
try:
    sql = """
        INSERT INTO InventoryItems
        (ItemName, Quantity, Category, ExpirationDate)
        VALUES (%s, %s, %s, %s)
    """

    cursor.execute(
        sql,
        (item_name, quantity, category, expiration_date)
    )
    conn.commit()
```

To edit an item:

```
        sql = """
        UPDATE InventoryItems
        SET ItemName = %s, Category = %s, Quantity = %s,
        ExpirationDate = %s
        WHERE ItemId = %s
        """
        cursor.execute(sql,(item_name,category,quantity,expiration_date,item_id))
        conn.commit()
        cursor.close()
        conn.close()

        return redirect(url_for('expiry_auditor'))
    cursor.execute(
        "SELECT * FROM InventoryItems where ItemId =%s",
        (item_id,)
    )
    item = cursor.fetchone()
```

To delete an item:

```
cursor.execute(
    "DELETE FROM InventoryItems where ItemId= %s",
    (item_id,)
)
```

The added inventory items can be seen in the expiry date auditor, which runs the following query:

```
sql = """
    SELECT ItemId, ItemName, ExpirationDate, Quantity, Category,
           DATEDIFF(ExpirationDate, CURDATE()) AS days_remaining,
           CASE
               WHEN Quantity<=10 THEN '!!RESTOCK NEEDED!!'
               ELSE 'Stock Available :)'
            END AS RestockStatus
    FROM InventoryItems
    ORDER BY ItemId ASC,ExpirationDate ASC
"""
cursor.execute(sql)
items = cursor.fetchall()
```

From the expiry date auditor, we can update our inventory stock:

```
cursor.execute(
    "SELECT Quantity FROM InventoryItems WHERE ItemId = %s",
    (item_id,)
)
item = cursor.fetchone()

if item:
    current_quantity = item['Quantity']

    if action== 'add':
        new_quantity = current_quantity + amount
    elif action== 'remove':
        new_quantity = current_quantity - amount

        if new_quantity<0 :
            new_quantity = 0
    else:
        new_quantity = current_quantity
    cursor.execute(
        "UPDATE InventoryItems SET Quantity =%s WHERE ItemId = %s",
        (new_quantity,item_id)
    )
conn.commit()
```

Moreover, one of my features had to manage warehouses and their corresponding inventory stock by shelf number. To view warehouses:

```
cursor.execute(
    "SELECT *FROM Warehouses order by WID ASC"
)

warehouses = cursor.fetchall()
```

To add/register a warehouse:

```
cursor.execute(
    """INSERT INTO Warehouses (Manager,Capacity,Contact)
        values (%s,%s,%s)
    """,(manager,capacity,contact)
)

conn.commit()
```

To edit warehouses:

```
        cursor.execute(
            """
            UPDATE Warehouses
            set Manager=%s, Capacity= %s, Contact= %s
            where WID = %s
            """,(manager,capacity,contact,wid)
        )
        conn.commit()
```

To delete a warehouse:

```
    cursor.execute(
        "DELETE from Warehouses where WID=%s",
        (wid,)
    )
```

To assign an inventory item to a warehouse:

```
    cursor.execute(
        """
        INSERT INTO warehouse_contains_inventoryitems (ItemId,WID, shelf_location)
        values (%s,%s,%s)
        """,
        (item_id,wid,shelf_location)
    )
```

The query below allows us to choose the inventory items from a (dropdown) list:

```
cursor.execute(
    """
    SELECT ItemId, ItemName
    from InventoryItems
    ORDER by ItemName ASC
    """
)
items = cursor.fetchall()
```

Finally, to view the inventory in a warehouse:

```
cursor.execute(
    """
    SELECT i.ItemId, i.ItemName, i.Category, i.Quantity,
    i.ExpirationDate, wci.shelf_location
    FROM warehouse_contains_inventoryitems wci
    JOIN inventoryitems i ON wci.ItemId = i.ItemId
    WHERE wci.WID = %s
""",(wid,)
)
items = cursor.fetchall()
```

Source Code Repository

Google Drive link with Project Source Code:

Google Drive:
https://drive.google.com/drive/folders/1jyuPn6trXx8PWMf4x6A1wl15ViQ7elGS?usp=drive_link
GitHub:
https://github.com/oishee296/CSE370_Group08_S11_Summer2026

Conclusion

Through this project, we learned how to handle a real-life problem using an RDBMS to create relationships between different entities and using SQL queries to store, retrieve and manage data. We also learned to connect our database with a Flask application and create features like registration, login, deployment, and updating. To sum up, our project helped us understand how to deal with the frontend, backend, and database all together to build a working system.

References